

ASSIGNMENT 4

1. Define Software Testing.

Software Testing is the process of evaluating a software application to verify that it behaves as expected, meets its requirements, and is free of defects. It involves executing the program with test inputs and comparing actual results with expected results. For this project, testing included automated unit testing (using Python's unittest) and manual functional testing to check that all 14 FAQ categories returned correct answers, that typo-tolerant matching worked, and that login/authentication behaved correctly for valid and invalid inputs.

2. What is Risk Identification?

Risk Identification is the process of recognizing potential problems that could negatively affect a project before they occur, so they can be planned for or mitigated. Examples relevant to this project include: the free hosting tier going to sleep and delaying responses (~30s cold start), the admin database not being updated regularly leading to outdated answers, or keyword-based matching failing to understand oddly phrased questions (mitigated using difflib fuzzy matching).

3. What is SCM?

SCM (Software Configuration Management) is the discipline of tracking and controlling changes to a software project's files over time, ensuring consistency and enabling collaboration. In this project, SCM was implemented using Git for version control and GitHub for hosting the repository, allowing every change to be tracked, committed with a message, and safely pushed to the live deployment on Render with automated continuous integration.

4. Define Debugging.

Debugging is the process of identifying, analyzing, and fixing errors (bugs) in a program's source code. It involves reproducing the faulty behavior, tracing it back to its root cause (often using logs, error messages, or print statements), and correcting the code so the program behaves as intended.

5. Explain Configuration Audit.

A Configuration Audit is a formal review conducted to verify that a software product conforms to its specified requirements and that all configuration items (code, documentation, versions) are consistent and correctly recorded. It typically has two parts: a Functional Configuration Audit (verifying the software meets its functional and performance requirements) and a Physical Configuration Audit (verifying the software matches its technical documentation).

6. Differentiate Between Black Box Testing And White Box Testing.

Black Box Testing	White Box Testing
Tests functionality without knowledge of internal code structure.	Tests internal code structure, logic, and paths directly.
Performed from the end-user's perspective (inputs and outputs).	Performed from a developer's perspective (code-level logic).
Example: checking the chatbot returns the right FAQ answer for a given query.	Example: checking the <code>match_query()</code> function's keyword-matching scoring logic directly.
Does not require programming knowledge.	Requires programming knowledge to design test cases.

7. What Do You Mean By Metrics?

Software Metrics are quantitative measures used to assess various aspects of a software project, such as its size, complexity, quality, or the productivity of the development process. Examples include lines of code, number of defects found, response time, or number of test cases passed (e.g. 7/7 automated unit tests passed in this project). In this application, live KPI metrics are also displayed on the admin dashboard showing Total Active FAQs, Total Queries Handled, and Registered Students.

8. What Do You Mean By Version Control?

Version Control is a system that records changes to files over time so that specific versions can be recalled, compared, or reverted later, and multiple people can collaborate without overwriting each other's work. This project uses Git for version control, with GitHub used to commit and push changes, and Render automatically redeploying the latest committed version.

9. Explain Object Oriented Software Engineering.

Object Oriented Software Engineering (OOSE) is an approach to designing and building software around “objects” — units that bundle data (attributes) and behavior (methods) together, based on real-world entities. It relies on concepts such as encapsulation, inheritance, and polymorphism. In this project, the database design reflects an object-oriented way of thinking about entities — Student, CollegeInfo, ChatHistory, and Admin — each with clearly defined attributes and methods.

10. Explain Various Types of Testing in details.

Unit Testing – Testing individual functions or components in isolation. Example: testing the `match_query()` function alone with sample inputs and fuzzy typos.

Integration Testing – Testing how multiple components work together. Example: testing that the Flask route correctly calls the database, logs chat history, and returns a JSON response.

System Testing – Testing the complete, integrated application as a whole. Example: using the live deployed chatbot end-to-end, from speaking a question to receiving an answer.

Acceptance Testing – Testing to confirm the system meets user requirements. Example: confirming a student can log in and see their correct attendance, CGPA, and mentor details.

Regression Testing – Re-testing existing features after a change, to confirm nothing that previously worked has broken. Example: re-testing FAQ answers and table rendering after adding dark mode and voice features.

Performance Testing – Testing how the system behaves under load or response-time constraints. Example: measuring API latency and documenting the free-tier hosting's cold-start delay after inactivity.

11. Differentiate Between Automated Testing And Manual Testing.

Manual Testing	Automated Testing
Test cases are executed by a human, step by step.	Test cases are executed by scripts/tools without human intervention.
Slower and more time-consuming for repeated tests.	Faster for repeated/regression testing once set up.
No programming/scripting knowledge required.	Requires writing test scripts (e.g. using a testing framework).
This is how the user experience was verified — manually testing voice input, speech output, dark mode, login, and admin panel on desktop and mobile browsers.	Implemented an automated test suite (test_suite.py) using Python's unittest framework to test Flask routes, keyword scoring, and student personalization logic with 100% pass rate.